

MANUAL TÉCNICO YFERA FRAMEWORK COMPILADORES 1



GeneradorEstilos:

La clase GeneradorEstilos actúa como el punto de entrada único para la transformación de código de estilos a CSS.

1. Información General

Tipo: Clase de Orquestación.

Responsabilidad: Coordinar el flujo de datos entre el Parser, el Analizador Semántico y el Traductor.

2. Atributos y Dependencias

Dependencias Externas:

parser (Jison): Motor encargado del análisis léxico y sintáctico.

AnalizadorSemanticoEstilos: Clase encargada de validar la lógica de los estilos.

TraductorEstilos: Clase encargada de generar el código CSS final.

3. Métodos Principales

analizar(entrada: String): Object

Es el método motor de la clase. Realiza las siguientes etapas:

Inicialización: Limpia los estados del parser (tokens y errores previos).

Análisis Sintáctico: Ejecuta parser.parse(entrada). Captura errores fatales mediante un bloque try-catch.

Análisis Semántico: Instancia el analizador y valida las definiciones obtenidas del AST.

Consolidación de Errores: Une los errores léxicos, sintácticos y semánticos en una sola lista.

Traducción Condicional: Si la lista de errores es igual a 0, procede a llamar al traductor para generar el CSS.

4. Estructura de Retorno

El método devuelve un objeto con la siguiente firma:

exito: (Boolean) true si no hubo errores de ningún tipo.

resultado: (Array) El AST (Abstract Syntax Tree) generado.

tablaSimbolos: (Object) La tabla de símbolos resultante del análisis.

css: (String) El código CSS generado (vacío si hubo errores).

errores: (Array) Lista de objetos ErrorYFERA encontrados.

5. Manejo de Errores Críticos

Posee una lógica de recuperación que, en caso de un error sintáctico no recuperable, intenta extraer la ubicación exacta (línea y columna) consultando el último token registrado por el parser antes del colapso.

Especificación Gramatical (estilos.json)

Este archivo define el Análisis Léxico (Lexer) y el Análisis Sintáctico (Parser) del lenguaje. Es el encargado de transformar el flujo de texto de entrada en una estructura de datos organizada (AST - Abstract Syntax Tree).

1. Información General

Tipo: Definición de Gramática (LALR).

Responsabilidad: Reconocer tokens, validar la estructura jerárquica del código y construir los nodos del AST.

2. Componentes del Lexer (Análisis Léxico)

El lexer utiliza expresiones regulares para identificar los componentes básicos del lenguaje:

Palabras Reservadas: @for, extends, from, through, to.

Símbolos: {, }, =, :, +, -, *, /, (,).

Identificadores:

VARIABLE: Identificada por el prefijo \$ (ej. \$miVar).

IDENTIFICADOR: Nombres de estilos o propiedades (admite guiones, ej. mi-estilo-1).

Valores Numéricos: Soporta ENTERO, DECIMAL y PORCENTAJE.

3. Componentes del Parser (Análisis Sintáctico)

La gramática es de tipo Recursiva por la Izquierda para facilitar el manejo de listas.

Punto de Entrada: archivo, el cual retorna un objeto conteniendo las definiciones y los errores acumulados.

Estructuras Soportadas:

Estilos: Soporta definición simple y herencia mediante la palabra clave extends.

Bucles (@for): Soporta dos variantes: through (inclusivo) y to (exclusivo).

Operaciones Aritméticas: Define precedencia de operadores (* / sobre + -) para permitir expresiones complejas en los valores de las propiedades.

4. Gestión de Errores e Integración

Recuperación de Errores: Utiliza el token especial error en reglas clave (estilos, bucles, propiedades) para evitar que el parser se detenga ante el primer fallo, permitiendo recolectar múltiples errores sintácticos.

Sincronización: Emplea la función marcarToken para rastrear la ubicación exacta (línea/columna) de cada símbolo procesado, facilitando el reporte de errores preciso en el objeto ErrorYFERA.

Acumulación: Los errores léxicos (caracteres no reconocidos) se agrupan de forma inteligente si son consecutivos para no saturar el reporte al usuario.

5. Estructura del AST Generado

El parser entrega una lista de objetos con tipos definidos:

{ tipo: 'estilo', nombre, extiende, propiedades, linea, columna }

{ tipo: 'for', variable, desde, hasta, inclusivo, cuerpo, linea, columna }

Tabla de Símbolos (TablaSimbolos)

La clase TablaSimbolos es una estructura de datos jerárquica que implementa el concepto de Ámbitos. Su función principal es almacenar información sobre los identificadores (estilos y variables) y resolver su alcance a lo largo del código.

1. Información General

Tipo: Estructura de Datos.

Responsabilidad: Almacenar, buscar y actualizar símbolos, gestionando la jerarquía de padres para permitir la herencia de ámbitos.

2. Estructura Interna

nombre: Identificador del ámbito actual (ej. 'global', 'for_\$i').

padre: Referencia a la instancia de la tabla superior. Si es null, se considera el ámbito raíz.

simbolos: Un Map de JavaScript que asocia el nombre del identificador con un objeto de información (metadatos del estilo o variable).

3. Funcionalidades Clave

A. Resolución en Cascada

El método buscar(nombre) implementa la búsqueda ascendente:

Busca el símbolo en el Map local.

Si no lo encuentra y existe un padre, delega la búsqueda hacia arriba.

Retorna null solo si llega al scope global y el símbolo no existe.

B. Gestión de Ámbitos Anidados

Mediante crearScopeHijo(nombreScope), la tabla facilita la creación de contextos

temporales (como el cuerpo de un bucle @for), vinculando automáticamente el nuevo scope con el actual como su padre.

C. Control de Unicidad Local

El método insertar(nombre, info) valida si el símbolo ya existe específicamente en el ámbito actual antes de agregarlo, lo que permite detectar redefiniciones de estilos en el mismo nivel.

4. Métodos de Inspección y Debug

obtenerTodos(): Aplana la jerarquía y devuelve una lista única de todos los símbolos visibles desde el punto actual, respetando el sombreado (shadowing) de variables si existiera.

5. Casos de Uso en el Proyecto

Registro de Estilos: Almacena las propiedades y la información de herencia de cada bloque identificado.

Validación de Variables: Registra las variables de control de los bucles @for para que solo sean accesibles dentro del cuerpo del bucle.

Suite de Validadores Semánticos

Esta suite de clases se encarga de asegurar que el AST generado por el parser sea lógicamente coherente, respetando las reglas de negocio del lenguaje de estilos (evitar ciclos, validar tipos de datos y manejar ámbitos).

1. Validador de Herencia (ValidadorHerencia)

Responsabilidad: Supervisar la integridad de las relaciones entre estilos que utilizan extends.

Detección de Auto-herencia: Bloquea casos donde un estilo intenta extenderse a sí mismo.

Validación de Existencia: Asegura que el estilo padre esté registrado en la TablaSimbolos.

Análisis de Ciclos: Implementa un recorrido para detectar herencias circulares (ej. A extiende a B, B extiende a A), evitando bucles infinitos en la traducción.

Regla de Reporte: Utiliza un criterio alfabético para reportar ciclos solo una vez, manteniendo la limpieza en el informe de errores.

2. Validador de Propiedades (ValidadorPropiedades)

Responsabilidad: Actuar como un motor de reglas de tipado para los pares clave-valor.

Diccionario de Reglas: Contiene un mapeo de propiedades CSS comunes (width, color, margin, etc.) hacia métodos de validación específicos.

Validación de Tipos:

Númericos: Verifica que valores para padding o margin sean enteros o decimales.

Mixtos: Permite porcentajes en propiedades como height o width.

Enumerados: Valida que text-align solo acepte CENTER, LEFT o RIGHT, y que las fuentes o estilos de borde estén dentro de la lista permitida.

Flexibilidad: Ignora la validación si el valor es una variable o una operación binaria, delegando esa resolución al tiempo de ejecución o traducción.

3. Validador de Bucles (ValidadorBucleFor)

Responsabilidad: Asegurar la validez de la estructura y el contenido de las iteraciones @for.

Validación de Rango: Comprueba que el límite inicial (desde) no sea mayor al límite final (hasta), reportando rangos invertidos.

Gestión de Ámbito Temporal: Crea un Scope Hijo en la TablaSimbolos donde registra la variable de iteración (ej. \$i). Esto permite que la variable sea "visible" solo dentro del bucle.

Delegación Recursiva: Instancia versiones locales de los validadores de herencia y propiedades para analizar el cuerpo del bucle utilizando el nuevo contexto de símbolos.

Analizador Semántico (AnalizadorSemanticoEstilos)

Es el componente de alto nivel que coordina a los validadores anteriores.

1. Información General

Responsabilidad: Realizar el análisis de múltiples pases sobre las definiciones del AST.

2. Lógica de Doble Pase

Para permitir que un estilo pueda heredar de otro que se define más abajo en el archivo (referencias hacia adelante), el analizador opera en dos etapas:

Pase de Registro: Recorre todas las definiciones y registra los nombres de los estilos en la TablaSimbolos. Si detecta un nombre duplicado en el mismo nivel, genera un error inmediato.

Pase de Validación: Una vez que la tabla conoce todos los estilos existentes, recorre nuevamente las definiciones aplicando los validadores de herencia, propiedades y bucles.

3. Estructura de Salida

Retorna un objeto consolidado con:

tabla: La instancia final de la TablaSimbolos (útil para depuración o herramientas de autocompletado).

errores: Una lista única que contiene todos los hallazgos de los validadores especializados.

Traductor de Estilos (TraductorEstilos)

El TraductorEstilos actúa como el motor de emisión de código. Su arquitectura está diseñada como una tubería de transformación (pipeline) que procesa las definiciones en etapas secuenciales para asegurar que el CSS resultante sea plano, válido y optimizado.

1. Información General

Responsabilidad: Transformar el AST en una cadena de texto (String) compatible con navegadores o preprocesadores CSS.

2. Pipeline de Traducción

El método principal traducir(definiciones) ejecuta cuatro pasos críticos:

Aplanamiento (_aplanarDefiniciones): Utiliza la clase ExpansorBucleFor para ejecutar la lógica de los bucles. Si encuentra un @for, genera múltiples bloques de estilo individuales, eliminando la estructura jerárquica del bucle.

Mapeo de Nombres: Crea un diccionario rápido (mapaEstilos) donde la clave es el nombre del estilo. Esto es esencial para el siguiente paso de resolución.

Resolución de Herencia: Delega en ResolutorHerencia la tarea de combinar las propiedades de un estilo padre con las del hijo, asegurando que el hijo sobrescriba las propiedades duplicadas.

Generación de String (_estiloACSS): Convierte los objetos JSON resultantes en bloques de texto con formato CSS (selectores, llaves y puntos y coma).

3. Funcionalidades Especiales

Sanitización de Selectores (`_asegurarNombreCSS`): Implementa una regla de seguridad que antepone un guion bajo (`_`) si un nombre de estilo comienza con un número, evitando que el CSS generado sea inválido según el estándar W3C.

Filtrado de Bloques Vacíos: El traductor descarta automáticamente cualquier estilo que no posea propiedades después de la resolución, manteniendo el archivo de salida limpio.

Delegación de Valores: Utiliza `MapeadorCSS` para traducir nombres de propiedades internas a nombres de propiedades CSS reales (por ejemplo, convertir `text size` a `font-size`).

4. Dependencias del Módulo

`ExpansorBucleFor`: Clase que resuelve la lógica matemática y de repetición del bucle.

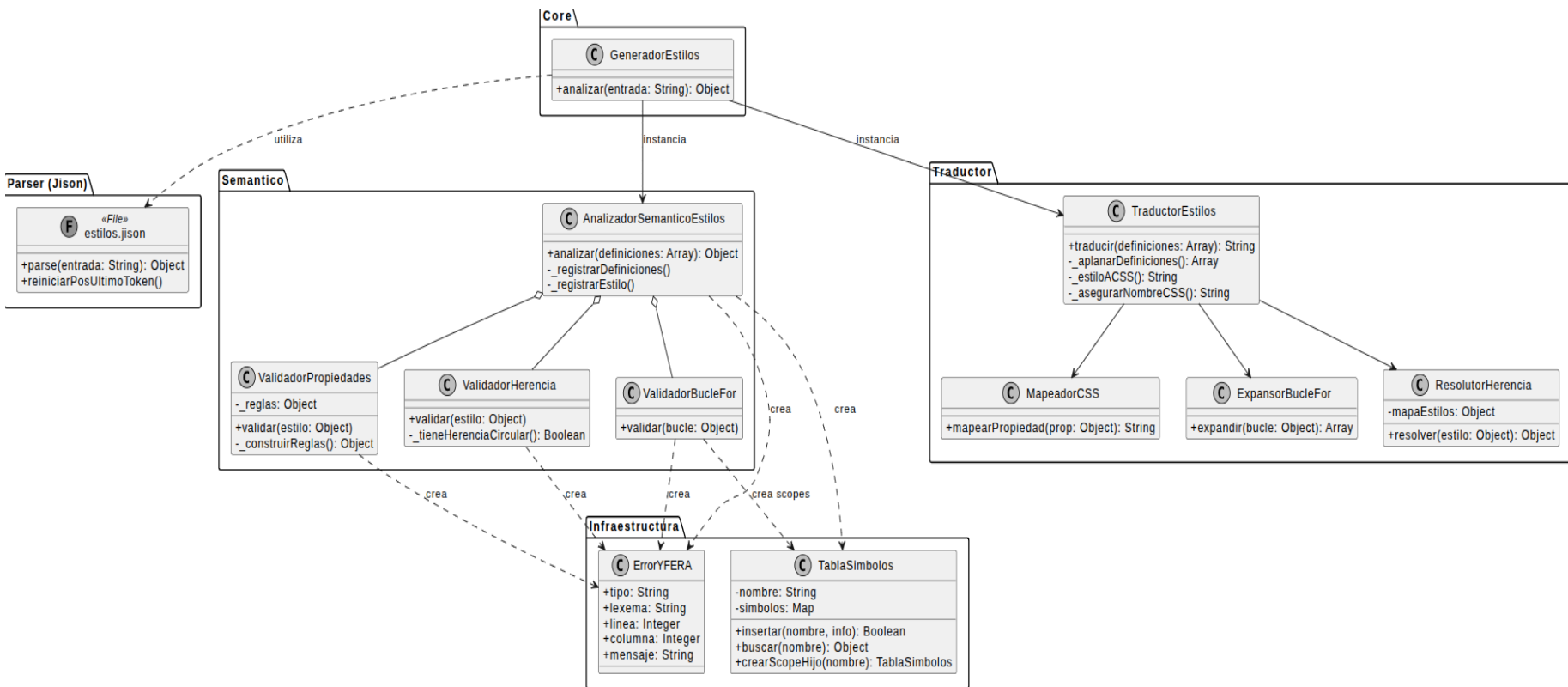
`ResolutorHerencia`: Clase que realiza el "merge" de propiedades entre clases relacionadas por `extends`.

`MapeadorCSS`: Diccionario encargado de la sintaxis final de cada línea de propiedad.

5. Estructura de Salida

El resultado es un String único donde cada bloque de estilo está separado por un doble salto de línea (`\n\n`), listo para ser guardado en un archivo `.css` o inyectado en un entorno web.

Diagrama de clases:



Lenguaje Comp

GeneradorComp

La clase `GeneradorComp` actúa como el punto de entrada único para la transformación del lenguaje de componentes a HTML. Implementa el patrón , encapsulando la interacción entre el parser de componentes, el motor semántico y el generador de marcado final.

1. Información General

Responsabilidad: Coordinar el flujo de datos entre el Parser de componentes, el Analizador Semántico y el Traductor a HTML.

2. Atributos y Dependencias

Dependencias Externas:

`parser (Jison)`: Motor encargado del análisis léxico y sintáctico específico de la gramática de componentes.

`AnalizadorSemanticoComp`: Clase encargada de validar ámbitos (scopes), existencia de variables y consistencia de tipos.

`TraductorComp`: Clase encargada de transformar el AST validado en estructuras HTML y metadatos JSON.

3. Métodos Principales

`analizar(entrada: String): Object`

Es el método motor de la clase. Coordina el pipeline de compilación:

Análisis Sintáctico: Ejecuta `parser.parse(entrada)`. Si el código fuente no respeta la gramática (ej. falta de llaves o etiquetas mal cerradas), captura la excepción y detiene el flujo.

Análisis Semántico: Toma el AST generado y lo entrega al `AnalizadorSemanticoComp`. En esta etapa se puebla la tabla de símbolos y se validan las referencias a variables (`$var`) y componentes.

Filtrado de Errores: Centraliza los errores detectados en las fases anteriores.

Generación de Salida: Si la lista de errores está vacía, invoca al `TraductorComp`. Este paso es crítico ya que el traductor asume que el AST es semánticamente correcto (todas las variables existen y los tipos son válidos).

4. Estructura de Retorno

El método devuelve un objeto con la siguiente firma:

`html: (String)` El código HTML completo resultante de la unión de todos los componentes procesados.

`componentes: (Object)` Diccionario donde la clave es el nombre del componente y el valor es su fragmento HTML individual.

`errores: (Array)` Lista de objetos `ErrorYFERA` encontrados durante el proceso (vacío si la operación fue exitosa).

`ast: (Object)` El árbol de sintaxis abstracta generado (útil para depuración).

5. Control de Flujo de Datos

A diferencia del generador de estilos, este orquestador maneja una salida dual (HTML global y componentes individuales), lo que permite al sistema principal decidir si inyecta el código completo en una página o si gestiona los componentes por separado según la lógica del frontend.

AnalizadorSemanticoComp (Coordinador Semántico)

Esta clase actúa como el cerebro lógico del compilador. Su función no es entender la estructura del código (de eso se encarga el Parser), sino validar que lo que el usuario escribió "tenga sentido" y respete las reglas de integridad del lenguaje.

1. Información General

Responsabilidad: Validar la integridad lógica del AST, gestionar la Tabla de Símbolos y coordinar a los validadores especializados.

2. Atributos y Dependencias

Dependencias Internas (Validadores):

ValidadorVariables: Motor principal de recorrido del AST y chequeo de existencia de variables.

ValidadorComponente: Encargado de validar las firmas y parámetros de los componentes.

ValidadorBucleFor, ValidadorCondicion, ValidadorFormulario: Validadores de estructuras de control y lógica de UI.

Estado:

tabla: Instancia de TablaSimbolos que representa el Scope Global.

errores: Array que recolecta los objetos ErrorYFERA generados en cualquier etapa del análisis.

3. Métodos Principales

analizar(ast: Array): Object

Es el punto de entrada al análisis semántico. Ejecuta un proceso en dos pasadas:

Primera Pasada (Registro Global): Recorre el AST buscando únicamente las declaraciones de componentes. Registra sus nombres y parámetros en la tabla de símbolos. Esto permite que los componentes puedan referenciarse entre sí sin importar el orden de declaración.

Segunda Pasada (Validación de Cuerpo): Delega en el ValidadorComponente el análisis profundo del contenido de cada componente (variables, lógica de control, formularios, etc.).

Gestión de Errores: Monitorea y consolida todos los fallos detectados por sus sub-validadores.

4. Lógica de Ámbitos (Scopes)

El analizador coordina la creación de scopes jerárquicos:

Scope Global: Contiene los nombres de todos los componentes definidos.

Scope de Componente: Contiene los parámetros y variables locales del componente.

Scopes Locales: Creados dinámicamente por bucles for o bloques lógicos, heredando las variables de los scopes superiores.

5. Estructura de Retorno

Devuelve un objeto que resume el estado de salud del código:

errores: (Array) Lista completa de inconsistencias encontradas (ej. "Variable \$x no declarada").

tablaSimbolos: (Object) La estructura final de símbolos y scopes procesados.

ValidadorVariables (Motor de Recorrido y Chequeo)

Esta clase es el motor recursivo del análisis semántico. Su función es recorrer cada rincón del Árbol de Sintaxis Abstracta (AST) para asegurar que cada uso de una variable, cada expresión matemática y cada llamada a función esté respaldada por una declaración válida en el ámbito (scope) correspondiente.

1. Información General

Tipo: Clase de Recorrido y Validación Cruzada.

Responsabilidad: Validar la existencia de variables en expresiones, gestionar la recursión por el AST y despachar nodos complejos a validadores específicos.

2. Atributos y Dependencias

Dependencias Externas:

TablaSimbolos: Utilizada para consultar la existencia de variables en el scope actual o superior.

ErrorYFERA: Clase para instanciar errores semánticos personalizados.

Referencias de Despacho (Setters):

validadorBucleFor: Referencia al validador de ciclos.

validadorCondicion: Referencia al validador de if/switch.

validadorFormulario: Referencia al validador de estructuras de formulario.

3. Métodos Principales

validarElementos(elementos: Array, scope: Object): void

Punto de entrada recursivo. Itera sobre una lista de nodos y decide cómo procesar cada uno según su tipo:

Nodos Contenedores (seccion, tabla): Re-invoca la recursión sobre sus hijos.

Nodos Lógicos (if, switch, for): Delega la validación a los objetos especializados, pasando el scope actual para mantener la jerarquía.

Nodos de Interacción (input, formulario): Valida las propiedades y referencias de ID.

validarExpresion(expr: Object, scope: Object): void

Realiza un recorrido de profundidad en expresiones (sumas, comparaciones, etc.). Si encuentra un nodo tipo variable, invoca a `_chequearVariable` para confirmar que el programador no está intentando usar un dato inexistente.

4. Lógica de Chequeo (`_chequearVariable`)

Es el método de "última milla". Realiza la consulta `scope.existe(nombre)`.

Si la variable no se encuentra en el scope actual ni en ninguno de sus padres (ascendiendo hasta el global), genera un nuevo ErrorYFERA con el mensaje: "La variable '\$nombre' no está declarada en este scope", incluyendo la línea y columna exacta del error.

5. Rol como Dispatcher (Despachador)

La clase implementa una Recursión Mutua:

ValidadorVariables detecta un for.

Llama a ValidadorBucleFor.

ValidadorBucleFor crea un nuevo scope hijo e inserta la variable iteradora.

ValidadorBucleFor llama de vuelta a ValidadorVariables.validarElementos para procesar el cuerpo del bucle con el nuevo scope.

ValidadorFormulario (Gestor de Integridad de UI)

Esta clase se especializa en validar la estructura y la lógica de los formularios dentro de los componentes. Su función es crítica para asegurar que la interacción entre los datos de entrada (inputs) y las acciones de envío (submit) sea coherente y esté libre de errores de referencia.

1. Información General

Responsabilidad: Asegurar la consistencia interna de los formularios, validando la unicidad de IDs y la correcta referencia de argumentos en las funciones de envío.

2. Atributos y Dependencias

Dependencias Externas:

ErrorYFERA: Para el registro de fallos semánticos en la estructura del formulario.

Estado Local:

inputsEncontrados: Un mapa o conjunto temporal que almacena los IDs de los inputs detectados dentro del formulario actual para validar referencias y duplicados.

3. Métodos Principales

validar(nodo: Object): void

Es el método principal que orquesta la revisión del formulario:

Recolección: Escanea los elementos hijos del formulario en busca de nodos tipo input.

Registro de IDs: Almacena los identificadores marcados con @id para permitir que el botón de submit los use como argumentos.

Validación de Submit: Si el formulario posee una acción submit, invoca a _validarReferenciasSubmit.

_validarReferenciasSubmit(submit: Object): void

Analiza la propiedad function: del botón de envío:

Verifica que, si la función recibe argumentos con el prefijo @ (ej. @nombre), dicho ID exista realmente en alguno de los inputs declarados dentro del mismo formulario.

Si se hace referencia a un ID inexistente, genera un ErrorYFERA indicando que el formulario intenta enviar un dato que no se está recolectando.

4. Reglas de Validación Específicas

Ámbito Cerrado: Las referencias @id son locales al formulario. Un formulario "A" no puede acceder a un input del formulario "B" mediante esta sintaxis.

Inputs Requeridos: Valida que las propiedades esenciales de los inputs (como el id cuando se usa en funciones) estén presentes.

Consistencia de Función: Asegura que la llamada a la función en el submit tenga una estructura de parámetros válida (literales, variables \$ o referencias @).

5. Integración en el flujo

Aunque el ValidadorVariables revisa que las variables \$ existan en la tabla de símbolos, el ValidadorFormulario es el único que tiene la "visión" completa de los elementos de interfaz para validar la conectividad entre los componentes visuales y la lógica de negocio.

ValidadorBucleFor (Gestor de Ámbitos Iterativos)

Esta clase se encarga de validar la lógica y la integridad de las estructuras repetitivas (for each y for complejo). Su función primordial es la gestión de la jerarquía de la Tabla de Símbolos, asegurando que las variables de iteración nazcan y mueran dentro del ciclo correcto.

1. Información General

Responsabilidad: Validar la sintaxis de los pares iteradores, verificar la existencia de los arreglos base y gestionar la creación de scopes hijos.

2. Atributos y Dependencias

Dependencias Externas:

TablaSímbolos: Para la creación del nuevo ámbito y la inserción de variables temporales.

ErrorYFERA: Para registrar errores de redeclaración o acceso a colecciones inexistentes.

Dependencias de Retorno (Rekursividad):

ValidadorVariables: Referencia necesaria para volver a validar el contenido (cuerpo) del bucle una vez establecido el nuevo contexto.

3. Métodos Principales

validar(nodo: Object, scopePadre: Object): void

Es el orquestador del ciclo de validación del bucle:

Validación de Colección: Comprueba que el arreglo sobre el cual se va a iterar (ej. \$productos) exista en el scopePadre.

Creación de Scope: Invoca scopePadre.crearScopeHijo("bucle_for") para aislar las variables del ciclo.

Registro de Iteradores: Extrae las variables definidas para la iteración (ej. \$item) e intenta insertarlas en el nuevo scope. Si el nombre ya existe en ese nivel, genera un error de redeclaración.

Procesamiento del Cuerpo: Llama a ValidadorVariables.validarElementos pasando el nuevo scope hijo para validar todo lo que está dentro del for.

`_validarPares(pares: Array, nuevoScope: Object): void`

Analiza la estructura variable : arreglo. Asegura que la variable de destino sea un identificador válido y que el arreglo sea una referencia a un símbolo existente.

4. Reglas de Ciclo y Ámbito

Ciclo de Vida: La variable iteradora solo es visible dentro de las llaves del for. Una vez finalizada la validación del nodo, ese scope se "cierra" para el resto del programa.

Soporte Multivariable: En el caso de los for complejos (múltiples arreglos), valida que todos los pares sean correctos antes de proceder al cuerpo.

Tratamiento de Índices: Si el bucle define un índice (track \$i), este también se registra en el scope local como un tipo numérico.

5. Prevención de Conflictos

El validador impide que el programador sobrescriba variables críticas del componente dentro de un bucle accidentalmente, ya que el sistema de TablaSimbolos detectará si se intenta declarar un iterador con un nombre que ya está siendo utilizado en el ámbito local inmediato.

ValidadorCondicion (Gestor de Lógica de Control)

Esta clase se especializa en la validación de las estructuras de ramificación del lenguaje (if, else if, else y switch). Su función es garantizar que las decisiones lógicas del programa se basen en expresiones válidas y que los tipos de datos involucrados sean coherentes con las reglas de control de flujo.

1. Información General

Tipo: Clase de Validación Lógica.

Responsabilidad: Validar que las condiciones de los bloques if sean booleanas y que las expresiones de un switch coincidan en tipo con sus respectivos case.

2. Atributos y Dependencias

Dependencias Externas:

InferenciaTipos: Herramienta estática fundamental para determinar el tipo resultante de una expresión compleja antes de validarla.

ErrorYFERA: Para registrar fallos cuando una condición no es booleana o un case es incompatible.

Estado:

tabla: Referencia a la tabla de símbolos para consultar variables dentro de las expresiones.

3. Métodos Principales

`validarIf(nodo: Object, scope: Object): void`

Evaluación de Condición: Solicita a InferenciaTipos el tipo de la expresión dentro del if.

Chequeo Booleano: Si el tipo no es boolean, registra un error. No se permiten "truthy/falsy" automáticos; la condición debe ser una comparación o un valor lógico explícito.

Recurción de Ramas: Valida de la misma forma todas las cláusulas else if presentes en el nodo.

`validarSwitch(nodo: Object, scope: Object): void`

Tipo Base: Evalúa la expresión principal del switch.

Restricción de Tipo: Valida que la expresión sea de tipo int o string, ya que el lenguaje restringe el control de flujo múltiple a estos tipos primitivos.

Consistencia de Casos: Recorre cada case y verifica que el valor literal coincida con el tipo de la expresión base (ej. si el switch es sobre un int, un case "texto" generará un error).

4. Reglas de Validación de Tipos

Inferencia Dinámica: Si la condición es una variable (ej. if(\$esValido)), el validador busca en la tabla de símbolos si \$esValido fue declarado como boolean.

Cortocircuito de Errores: Si InferenciaTipos devuelve desconocido (debido a un error previo en la expresión), el validador omite la validación de tipo para evitar una cascada de mensajes de error redundantes.

5. Manejo de Bloques

A diferencia del for, los bloques if y switch en este lenguaje no crean automáticamente un nuevo scope para sus elementos internos (las variables declaradas fuera siguen siendo las mismas), por lo que el validador utiliza el scope actual del componente para todas las verificaciones de contenido.

InferenciaTipos (Motor de Evaluación de Tipos)

Esta es una clase de utilidad estática que actúa como el motor analítico del compilador. Su función no es validar estructuras, sino "predecir" o calcular el tipo de dato resultante de cualquier expresión (aritmética, lógica o de acceso) basándose en las reglas de precedencia y promoción de tipos del lenguaje YFERA.

1. Información General

Tipo: Clase de Servicio Estática (Utility Class).

Responsabilidad: Determinar el tipo de dato de una expresión y gestionar la compatibilidad entre operandos.

2. Atributos y Dependencias

Dependencias Externas:

TablaSímbolos: Consultada exclusivamente para obtener el tipoDato de las variables declaradas cuando se encuentran dentro de una expresión.

Tipos Soportados:

int, float, string, boolean, char, function, desconocido.

3. Métodos Principales

static inferir(expr: Object, scope: Object): String

Es el método principal recursivo. Evalúa el nodo de la expresión y devuelve una cadena con el tipo:

Literales: Devuelve el tipo directo (ej. "hola" -> string, 10.5 -> float).

Variables: Busca el nombre en el scope y retorna su tipoDato.

Operaciones Aritméticas: Evalúa los hijos izquierdo y derecho, y aplica reglas de combinación.

Operaciones Lógicas/Comparación: Devuelve siempre boolean si los operandos son válidos.

`static _combinarNumericos(t1: String, t2: String): String`

Implementa la lógica de Promoción de Tipos:

Si ambos son int, el resultado es int.

Si uno es int y el otro float, el resultado "asciende" a float para evitar pérdida de precisión.

Si alguno es desconocido, el resultado es desconocido.

4. Lógica de Evaluación de Expresiones

La inferencia se realiza de abajo hacia arriba (Bottom-Up):

Se llega a las hojas (valores constantes o variables).

Se resuelve el tipo de los operandos.

Se aplica la regla del operador (ej. una suma de string e int podría resultar en un error o un tipo desconocido según la especificación del lenguaje).

5. Importancia en el Sistema

Esta clase es el punto de apoyo para otros validadores:

ValidadorCondicion la usa para saber si puede permitir un if.

ValidadorVariables la usa para verificar que una asignación sea válida.

TraductorComp la usa para decidir cómo formatear un valor en el HTML final.

TraductorComp (Generador de Marcado y Runtime)

La clase TraductorComp es el componente final del pipeline. Su responsabilidad es transformar el Árbol de Sintaxis Abstracta (AST), ya validado semánticamente, en código HTML compatible con navegadores, inyectando además metadatos lógicos para que el sistema sea interactivo en el cliente.

1. Información General

Tipo: Clase de Transformación y Generación de Código.

Responsabilidad: Generar el marcado HTML, resolver placeholders de variables y exportar la metadata de los formularios.

2. Atributos y Dependencias

Dependencias de Soporte:

InferenciaTipos: Utilizada para resolver tipos en expresiones complejas que deben ser impresas como texto.

Salida de Datos:

HTML: Estructura visual basada en etiquetas estándar y clases prefijadas (yfera-).

Metadata (JSON): Bloques de configuración incrustados para el manejo de eventos en el frontend.

3. Métodos Principales

`traducir(componentes: Array): Object`

Orquestador de la fase de salida. Recorre la lista de componentes validados y genera un diccionario de strings HTML, además de un consolidado global.

`_traducirElemento(nodo: Object): String`

Funciona como un despacho (switch) que convierte nodos del AST en etiquetas HTML:

seccion / tabla: Se transforman en <div> con clases CSS o estructuras <table> / <tr> / <td>.

texto: Procesa el contenido para convertir sintaxis como \$variable en placeholders tipo {{ \$variable }}.

imagen: Decide si renderizar una etiqueta simple o una estructura compleja de carrusel si existen múltiples URLs.

_extraerMetaSubmit(submit: Object): Object | null

Genera un objeto de configuración para los botones de envío. Identifica qué argumentos son literales, cuáles son variables del componente y cuáles son referencias @id que deben ser capturadas del DOM en tiempo de ejecución.

4. Sistema de Placeholders y Escapado

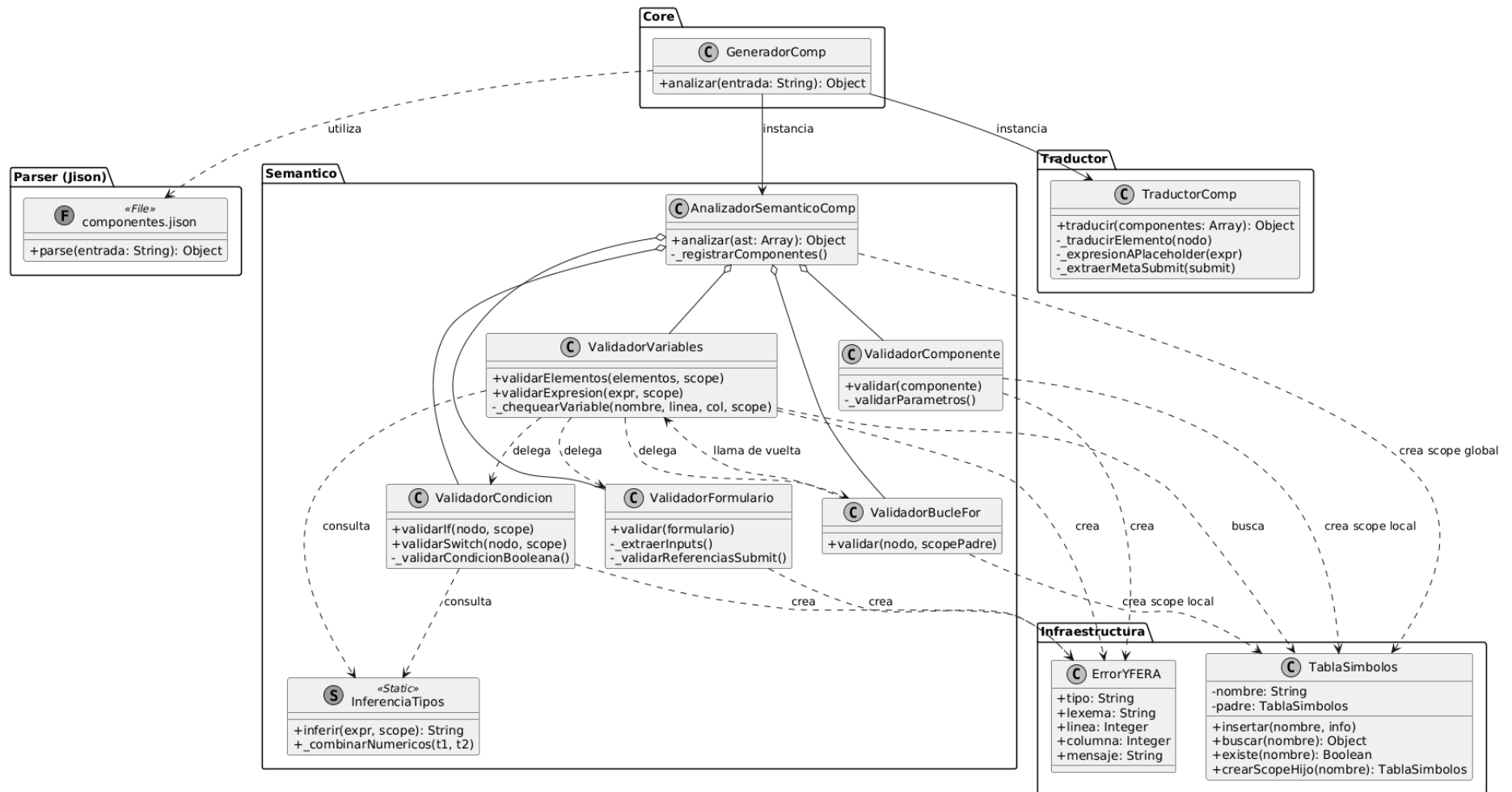
Interpolación: El traductor convierte la lógica del servidor a un formato de plantillas. Por ejemplo, una expresión de suma en el AST se traduce a: {{ \$valor1 + \$valor2 }}.

Seguridad: Implementa el método _escaparHtml para prevenir ataques de inyección (XSS) al renderizar valores literales o nombres de variables en atributos sensibles como value o src.

5. Manejo de Estilos Dinámicos

El traductor mapea el arreglo de estilos presente en cada nodo hacia el atributo class de HTML. Utiliza una convención de nombres donde siempre se incluye una clase base (ej. yfera-input) seguida de los estilos personalizados definidos por el usuario, permitiendo una integración limpia con el GeneradorEstilos.

Diagrama de clases Comp



Lenguaje Y

Analizador (Jison) - Procesador Central

Este archivo define la gramática formal del lenguaje utilizando Jison. Se encarga de transformar el código fuente en un Árbol de Sintaxis Abstracta (AST) detallado, manejando la recuperación de errores y la construcción de nodos para declaraciones, funciones y lógica de control.

1. Información General

Tipo: Generador de Analizador Léxico y Sintáctico (LALR).

Responsabilidad: Tokenizar la entrada, validar la estructura gramatical y generar el AST para su posterior análisis semántico.

2. Componentes Léxicos (Lexer)

El lexer maneja un estado especial para cadenas de texto y soporta los siguientes elementos:

Palabras Reservadas: import, function, main, execute, load, tipos de datos (int, float, etc.) y estructuras de control.

Tokens Especiales:

REFERENCIA: Identificadores que inician con @ (usados para invocación de componentes).

CADENA_SQL: Texto encerrado en comillas invertidas (`) para consultas directas.

CADENA: Soporta secuencias de escape como \n, \t y \".

Comentarios: Soporta tanto # (línea) como /* ... */ (multilínea).

3. Estructura del AST (Producciones)

El parser construye objetos JSON estandarizados con propiedades tipo, linea y columna.

Declaraciones: Maneja import, variables (simples y arreglos) y funciones.

Inicializadores de Arreglo: Soporta tres tipos: tamaño fijo [N], lista de valores {v1, v2} y resultados de base de datos EXECUTE "SQL".

Funciones y Main:

function_decl: Permite parámetros (simples o arreglos) y un cuerpo limitado a execute y load.

main_decl: El punto de entrada que permite toda la lógica de control (if, while, for, switch) e invocación de componentes.

Expresiones: Implementa la jerarquía de operadores (OR > AND > Comparación > Aritmética > Unaria > Primaria).

4. Recuperación de Errores

El analizador implementa estrategias de recuperación para no detener el proceso ante el primer fallo:

A nivel de lista: Si una declaración falla, el parser busca el siguiente PUNTO_COMA (;) para intentar procesar la declaración posterior.

Registro Centralizado: Los errores léxicos se capturan en el bloque . (punto) del lexer, mientras que los sintácticos se recolectan mediante la función registrarErrorSintactico.

5. Reglas de Invocación de Componentes

Una característica clave en esta gramática es la producción invocacion_componente. Utiliza el token REFERENCIA (ej: @MiComponente) seguido de argumentos, lo que vincula este lenguaje imperativo con el sistema de componentes visuales definido previamente.

GeneradorY (Orquestador del Pipeline de Compilación)

La clase GeneradorY actúa como el cerebro de la compilación para el lenguaje YFERA. Es un componente que encapsula la complejidad de las tres etapas principales: el análisis léxico/sintáctico (Jison), el análisis semántico y la traducción final a código ejecutable o HTML.

1. Información General

Tipo: Clase Orquestadora (Controller).

Responsabilidad: Gestionar el flujo de datos entre el parser, el validador semántico y el generador de código, consolidando los errores de todas las fases.

2. Atributos y Dependencias

yModulo: Referencia al analizador generado por Jison.

AnalizadorSemanticoY: El motor que valida la lógica, tipos de datos y ámbitos de las variables.

TraductorY: El encargado de generar la salida final (HTML/JS) basada en el contexto validado.

ErrorYFERA: Clase base para la estandarización de reportes de fallos.

3. Flujo de Ejecución (Método analizar)

El proceso se divide en cuatro fases críticas:

Configuración de Errores Sintácticos: Sobrescribe el método parseError de Jison para capturar errores de sintaxis de forma amigable, traduciendo los tokens esperados a mensajes claros para el desarrollador.

Fase de Parsing: Ejecuta yModulo.parse(entrada) para obtener el Árbol de Sintaxis Abstracta (AST). Si falla en esta etapa, los errores léxicos y sintácticos se recolectan inmediatamente.

Análisis Semántico: Solo si el AST es válido, instancia al AnalizadorSemanticoY. Aquí se verifica que las variables existan, los tipos coincidan y las funciones se llamen correctamente.

Fase de Traducción: Se ejecuta únicamente si hay cero errores semánticos. El TraductorY toma el contexto resultante y el AST para producir el HTML final.

4. Gestión Centralizada de Errores

Una de las mayores virtudes de esta clase es la unificación de errores. Al final del proceso, combina:

Errores del Lexer (caracteres inválidos).
Errores del Parser (mala estructura).
Errores Semánticos (lógica e integridad).
Errores de Traducción (fallos en tiempo de generación, como consultas SQL fallidas).

5. Resultado del Proceso

El método analizar retorna un objeto con una estructura limpia para el frontend o el servidor:
éxito: Booleano que indica si el proceso terminó sin ningún tipo de error.

ast: El árbol generado (útil para debugging).

tablaSimbolos: El estado final de la memoria del programa analizado.

html: El código generado listo para renderizar.

errores: Un array unificado con la ubicación exacta (línea/columna) de cada problema.

AnalizadorSemanticoY (Coordinador de Validación Imperativa)

Esta clase es la responsable de garantizar la integridad lógica del lenguaje principal. A diferencia de los validadores visuales, esta clase gestiona un flujo de ejecución imperativo, coordinando la validación de importaciones, memoria global y estructuras de control.

1. Información General

Tipo: Clase de Coordinación Semántica.

Responsabilidad: Clasificar los nodos del AST y delegar su validación a motores especializados, gestionando una única Tabla de Símbolos y un contexto de proyecto.

2. Atributos y Dependencias

tabla (TablaSimbolos): El almacén de memoria global donde se registrarán variables y funciones.

contexto (ContextoProyecto): El enlace con el sistema de archivos y otros módulos (Estilos y Componentes) importados.

Validadores Especializados:

ValidadorImports: Gestiona la resolución de dependencias externas.

ValidadorVariables: Valida declaraciones globales.

ValidadorFunciones: Valida la estructura y parámetros de las funciones.

ValidadorMain: Valida el punto de entrada y la interacción con componentes.

3. Métodos Principales

analizar(ast: Array): Object

Realiza un proceso de filtrado y validación secuencial:

Clasificación: Divide el AST en cuatro grupos: imports, variables, funciones y mains.

Validación de Dependencias: Ejecuta primero los imports para poblar el contexto con componentes y estilos externos.

Registro de Memoria: Valida y registra variables y funciones globales.

Validación de Ejecución: Valida el bloque main, permitiendo que este consulte el contexto para verificar si las invocaciones a componentes (ej. @MiComponente) son válidas.

4. Estructura de Retorno

Devuelve un objeto consolidado:

tabla: La tabla de símbolos resultante con todas las declaraciones procesadas.

errores: Una lista unificada de todos los errores semánticos encontrados en las fases de validación.

ContextoProyecto (Gestor de Dependencias y Sistema de Archivos)

Esta clase actúa como un puente entre el compilador y el entorno del proyecto. Su función es resolver rutas de archivos, leer contenidos y servir como un "repositorio central" de los elementos (estilos y componentes) que han sido importados.

1. Información General

Tipo: Clase de Infraestructura y Gestión de Cache.

Responsabilidad: Resolver dependencias externas, cargar y analizar archivos .yfera o .styles, y mantener un mapa de elementos disponibles para el compilador principal.

2. Atributos y Dependencias

cache (Map): Almacena los resultados de archivos ya analizados para evitar re-procesamientos costosos.

componentes / estilos (Maps): Diccionarios que guardan la ubicación (archivo, línea, columna) de cada elemento definido en el proyecto.

Motores Externos: Instancia a GeneradorEstilos y GeneradorComp para analizar el contenido de los archivos importados.

3. Métodos Principales

cargarStyles / cargarComp(rutaRelativa: String): Object

Es el núcleo de la resolución de dependencias:

Resolución de Ruta: Calcula la ruta absoluta basada en la ubicación del archivo actual y la raíz del proyecto.

Análisis Recursivo: Lee el archivo, invoca al generador correspondiente y extrae los nombres de los estilos o componentes definidos.

Registro: Guarda en los mapas globales la existencia de estos elementos para que el ValidadorMain pueda verificar invocaciones más adelante.

Cache: Almacena el resultado (incluyendo el CSS o HTML generado) para su uso en la fase de traducción.

buscarComponente / buscarEstilo(nombre: String): Object

Permite a los validadores realizar consultas rápidas para saber si un elemento invocado existe en el universo del proyecto, devolviendo su metadata.

4. Seguridad y Resolución de Rutas

Implementa una validación de seguridad en resolverRuta: asegura que el compilador nunca intente leer archivos fuera de la carpeta del proyecto (rutaProyecto), previniendo accesos no autorizados al sistema de archivos mediante rutas relativas malintencionadas.

5. Integración con el Traductor

Además de validar, la clase provee métodos como obtenerCssDelImport y obtenerTemplateComponente, permitiendo que el traductor final ensamble todas las piezas del rompecabezas en un único documento HTML.

ValidadorBloques (Motor de Lógica y Flujo)

Esta clase es la responsable de validar la estructura y coherencia de las sentencias dentro de los bloques de código (cuerpos de funciones, ciclos y condiciones). Su función principal es asegurar que las reglas de control de flujo y los tipos de datos en expresiones lógicas se respeten estrictamente.

1. Información General

Tipo: Clase de Validación de Sentencias.

Responsabilidad: Validar el cuerpo de los bloques, gestionar la profundidad de ciclos/sentencias y realizar inferencia de tipos básica.

2. Atributos de Control de Estado

La clase utiliza contadores para validar el contexto de sentencias especiales:

profundidadCiclo: Incrementa al entrar en un while, do-while o for. Permite validar que continue y break se usen solo donde es legal.

profundidadSwitch: Incrementa al entrar en un switch. Permite validar el uso de break fuera de ciclos pero dentro de casos.

3. Métodos de Validación de Estructuras

_validarIf / _validarSwitch(nodo, tabla, validadorMain):

If: Fuerza que la condición sea estrictamente boolean. Gestiona la lógica de ramas else if y asegura que no existan ramas adicionales después de un else final.

Switch: Valida que la expresión base sea numérica o string. Verifica que no existan valores duplicados en los case y que los tipos de los casos sean compatibles con la expresión del switch.

_validarWhile / _validarDoWhile / _validarFor:

Aseguran que las condiciones de parada sean booleanas.

En el caso del For, valida además la inicialización y que los operadores de incremento (++) o decremento (--) se apliquen exclusivamente a variables de tipo int o float.

4. Inferencia de Tipos (_inferirTipo)

Implementa un motor de resolución de tipos para expresiones:

Primitivos: Identifica int, float, string, char y boolean de literales.

Operaciones: Resuelve el tipo resultante de operaciones aritméticas (aplicando promoción a float si un operando lo es) y lógicas (siempre retornando boolean).

Identificadores: Consulta la TablaSimbolos para obtener el tipo de las variables o arreglos utilizados en la expresión.

5. Reglas de Sentencias de Salto

Break: Solo permitido si profundidadCiclo > 0 O profundidadSwitch > 0.

Continue: Solo permitido si profundidadCiclo > 0. El uso de un continue dentro de un switch que no está dentro de un ciclo se reporta como error semántico.

6. Manejo de Errores

Cada fallo (como una condición no booleana o una variable no declarada en un incremento) genera una instancia de ErrorYFERA con la ubicación exacta, la cual se concatena a la lista global de errores del análisis.

ValidadorFunciones (Integridad de Subrutinas)

Esta clase se encarga de procesar las declaraciones de funciones en el lenguaje YFERA, asegurando que su firma (nombre y parámetros) sea única y que su contenido cumpla con las restricciones de carga dinámica.

1. Información General

Tipo: Validador de Declaraciones.

Responsabilidad: Validar la no redundancia de funciones y parámetros, y verificar la sintaxis de las sentencias load dentro del cuerpo.

2. Reglas de Validación

Redeclaración: Verifica en la TablaSimbolos que el nombre de la función no haya sido utilizado previamente por otra función o variable global.

Parámetros: Utiliza un Set para garantizar que no existan nombres de parámetros duplicados en la misma firma (ej: function sumar(int a, int a) es inválido).

Restricción de Load: El lenguaje YFERA permite la carga dinámica de código mediante load. Este validador fuerza que, si se usa un literal (string directo), este deba tener obligatoriamente la extensión .y.

3. Registro en Tabla de Símbolos

Si la función es válida, se inserta en la tabla con la categoría funcion. Se guarda un mapa detallado de sus parámetros (nombre, tipo y si es arreglo), lo cual es crucial para validar las llamadas a funciones en etapas posteriores (comprobación de aridad y tipos).

ValidadorImports (Gestor de Dependencias Externas)

El ValidadorImports es un componente crítico que interactúa directamente con el sistema de archivos a través del ContextoProyecto. Su misión es asegurar que todas las dependencias externas (.styles y .comp) sean válidas y seguras.

1. Información General

Tipo: Validador de Infraestructura / Sistema de Archivos.

Responsabilidad: Validar extensiones, rutas relativas, existencia física de archivos y seguridad perimetral del proyecto.

2. Políticas de Seguridad y Formato

Extensiones Permitidas: Solo se aceptan archivos .styles (CSS especializado) y .comp (Componentes visuales).

Rutas Relativas: Fuerza el uso de ./ o ../. Esto evita el uso de rutas absolutas que romperían la portabilidad del proyecto entre diferentes máquinas.

Sandbox (PROYECTO): Gracias al método `resolverRuta` del contexto, el validador detecta si un `import` intenta "escapar" de la carpeta raíz del proyecto mediante el uso excesivo de `../`, bloqueando la operación por seguridad.

3. Validación de Integridad (Carga en Cascada)

A diferencia de otros validadores, este es recursivo en su lógica:

Si el archivo existe, solicita al `ContextoProyecto` que lo analice.

Si el archivo importado tiene errores (léxicos o sintácticos), el validador marca el `import` como fallido en el archivo actual.

Resultado: Se garantiza que un archivo `.y` solo compile si todas sus dependencias están 100% limpias de errores.

4. Registro de Dependencias

Los `imports` se registran en la tabla de símbolos utilizando su ruta como identificador. Esto permite que el compilador sepa qué recursos están disponibles para ser usados por el Traductor al momento de generar el HTML y CSS final.

ValidadorMain (Orquestador de Ejecución)

El `ValidadorMain` es el componente que asegura la existencia y unicidad del punto de entrada del programa. Además, actúa como el puente entre la lógica imperativa y el renderizado de componentes visuales.

1. Información General

Tipo: Validador de Punto de Entrada.

Responsabilidad: Garantizar que exista exactamente un bloque `main`, validar la asignación de variables y la correcta invocación de componentes externos.

2. Reglas de Estructura Global

Existencia: Lanza un error si no se encuentra el bloque `main`.

Unicidad: Si detecta más de un bloque `main`, marca los excedentes como errores semánticos (solo el primero es válido).

Cuerpo: Delega la validación de las sentencias internas a `ValidadorBloques`, pasando una referencia de sí mismo (`this`) para resolver invocaciones y asignaciones.

3. Validación de Invocaciones (`_validarInvocacion`)

Es la funcionalidad más importante para la integración de componentes:

Existencia de Componentes: Consulta al `ContextoProyecto` para verificar si un componente invocado (ej. `@Card`) fue realmente importado desde un archivo `.comp`.

Aridad (Parámetros): Compara la cantidad de argumentos enviados en la invocación contra la definición original del componente.

4. Validación de Asignaciones y Expresiones

Contexto de Memoria: Verifica que cualquier variable utilizada en una asignación o expresión exista en la `TablaSimbolos`.

Integridad de Arreglos: Asegura que solo se usen índices `[]` en variables declaradas como arreglos y viceversa.

ValidadorVariables (Gestor de Memoria Global)

Esta clase se encarga de procesar las declaraciones de variables globales, asegurando que no haya colisiones de nombres y que los valores iniciales coincidan con los tipos declarados.

1. Información General

Tipo: Validador de Declaraciones de Memoria.

Responsabilidad: Validar tipos de datos, inicialización de escalares y arreglos, y registro en la tabla de símbolos.

2. Compatibilidad de Tipos

Implementa una política de tipado fuerte con una excepción de promoción numérica:

Regla de Oro: El tipo recibido debe ser igual al esperado.

Promoción: Se permite asignar un int a una variable float, pero no viceversa (evita pérdida de precisión implícita).

3. Inicialización de Arreglos

El validador maneja dos formas de inicialización:

Por Tamaño (arreglo_tamano): Verifica que el tamaño definido sea un número entero no negativo.

Por Valores (arreglo_valores): Recorre cada elemento de la lista inicializadora y comprueba que todos coincidan con el tipo base del arreglo (ej: que no haya un string dentro de un arreglo de int).

4. Registro y Persistencia

Una vez validada, la variable se inserta en la TablaSimbolos con su metadata completa (tipoDato, esArreglo, ubicación), permitiendo que el resto de los validadores (como ValidadorMain) puedan realizar comprobaciones semánticas precisas.

TraductorY

El TraductorY es un motor de renderizado estático que transforma la lógica de archivos .y en una página web completa. Implementa un evaluador de expresiones en tiempo de compilación para resolver valores constantes y lógica de control simple.

1. Información General

Tipo: Compilador de Salida / Generador de Código.

Responsabilidad: Generar el archivo HTML final, inyectar CSS, resolver componentes y adjuntar un Runtime de JavaScript para la interactividad.

2. Mecanismo de Evaluación Estática

El traductor mantiene un Map de variables para simular un estado de memoria:

Recolectar Variables: Antes de traducir, recorre las declaraciones globales. Si una variable tiene un valor conocido (literales o SQL execute), lo guarda.

Evaluación de Expresiones (`_evaluarExpresion`): Resuelve operaciones matemáticas, lógicas y concatenaciones. Si una expresión depende de un valor desconocido (como una entrada de usuario futura), devuelve null y el traductor marca la sección como "no evaluable estáticamente".

3. Integración con SQL (`_ejecutarSQLGlobal`)

Es una de las características más potentes:

Permite inicializar arreglos usando sentencias SQL reales mediante la instrucción `execute`.

Se comunica con el `GeneradorSQL` del sistema para realizar consultas `SELECT` y traer datos de la base de datos del proyecto directamente al renderizado del HTML.

4. Renderizado de Componentes e Inyecciones

Invocación de Componentes (`_traducirInvocacion`):

Busca el template del componente (HTML con placeholders tipo `{{ $param }}`).

Evalúa los argumentos pasados en el archivo `.y`.

Sustituye los placeholders por los valores reales, escapando caracteres especiales para evitar ataques XSS.

Construcción de CSS:

Recolecta todos los archivos `.styles` importados y concatena su código CSS en una sola etiqueta `<style>` dentro del `<head>`.

5. El Runtime Script (`_generarRuntimeScript`)

El traductor inyecta un pequeño fragmento de JavaScript puro al final del HTML. Este script tiene tres misiones:

Interceptar Formularios: Evita que los formularios recarguen la página de forma tradicional.

Comunicación con el Servidor: Envía los datos de los formularios vía `fetch` a un endpoint de API para ejecutar funciones del lado del servidor.

Refresco de UI: Escucha la respuesta del servidor y, si es necesario, envía un `postMessage` al entorno (posiblemente un IDE o visor en tiempo real) para actualizar la vista.

6. Manejo de Estructuras de Control en Traducción

If/Switch: Si la condición se puede evaluar (ej: `if (10 > 5)`), el traductor solo genera el HTML de la rama verdadera. Si no se puede evaluar, incluye el código de todas las ramas envuelto en comentarios de advertencia.

Ciclos: Actualmente realiza una renderización de "una sola iteración" como previsualización estática.

InvocadorFuncionY (Runtime del Lado del Servidor)

El `InvocadorFuncionY` actúa como el endpoint de ejecución dinámica. Mientras que el `TraductorY` se encarga de la parte visual, esta clase se encarga de la interactividad: procesa formularios, ejecuta comandos SQL y gestiona la navegación entre archivos `.y`.

1. Información General

Tipo: Motor de Ejecución en Tiempo de Respuesta (RPC - Remote Procedure Call).

Responsabilidad: Validar argumentos enviados desde el cliente, sustituir variables en sentencias SQL, ejecutar comandos en la base de datos y controlar la recarga de la interfaz.

2. Proceso de Invocación (Ciclo de Vida)

Cuando el usuario presiona "Enviar" en un formulario generado por YFERA, ocurre lo siguiente:

Re-análisis: El invocador vuelve a parsear el archivo .y para obtener el AST actualizado de la función.

Mapeo de Tipos: Los datos de los formularios llegan como strings. El método `_convertirTipo` realiza el casting necesario (int, float, boolean) para que coincidan con la firma de la función.

Sustitución de Variables: Busca identificadores con el prefijo \$ en las sentencias SQL y los reemplaza por los valores reales del formulario, aplicando escape de comillas para prevenir inyecciones SQL.

Ejecución de Cuerpo: Se ejecutan secuencialmente las instrucciones `execute` y `load`.

3. El Comando `execute` (Interacción con BD)

Es el puente con el `GeneradorSQL`:

Toma el SQL ya procesado (con los valores inyectados).

Instancia el `GeneradorSQL` en modo `ejecutar: true`.

Captura el éxito o error de la operación para informarlo al frontend mediante alertas o mensajes de consola.

4. El Comando `load` (Navegación Dinámica)

Esta es la instrucción que permite crear aplicaciones multi-página:

Si la función ejecuta un `load "otro.y"`, el invocador devuelve una bandera `recargar: true`.

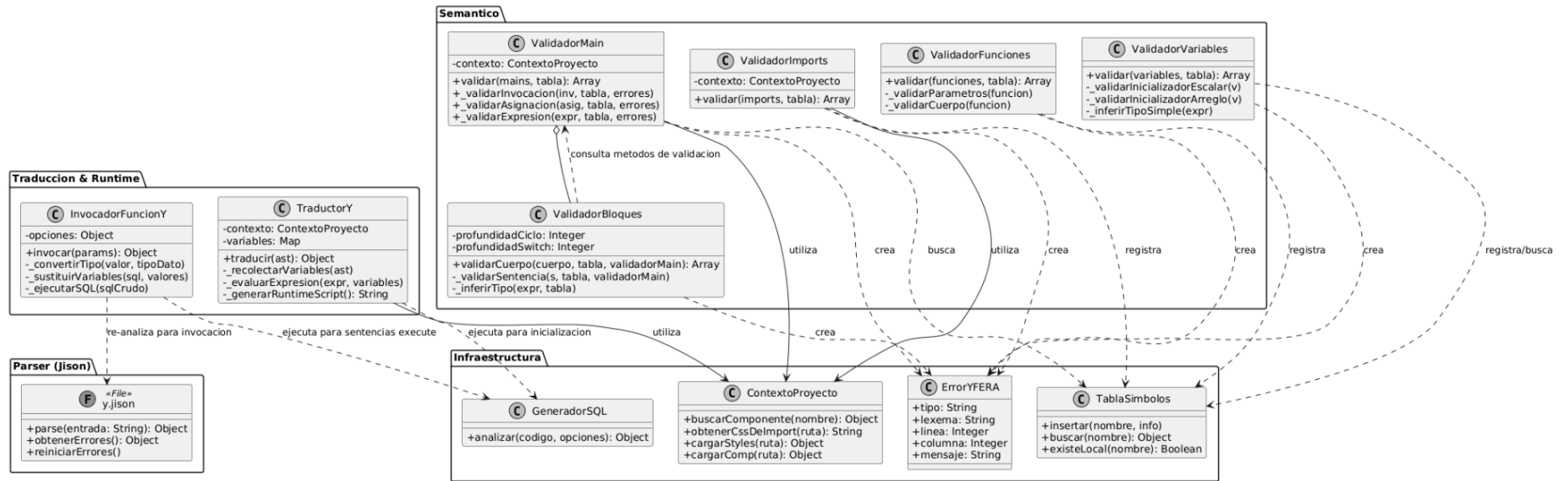
El script inyectado en el navegador captura esta bandera y fuerza una recarga de la página o notifica al IDE para cambiar de archivo, permitiendo flujos de usuario complejos.

5. Sistema de Conversión y Seguridad

Cast Seguro: Convierte valores de checkbox ("`true`"/"`false`") a booleanos reales y entradas de texto a números.

Sustitución Inteligente: Al sustituir variables en SQL, diferencia automáticamente entre tipos: envuelve los strings en comillas dobles y convierte booleanos a 0/1 para compatibilidad con bases de datos como SQLite.

Diagrama de clase Y



Lenguaje SQL

sql.jison (Motor de Persistencia)

Este archivo define el analizador léxico y sintáctico para el lenguaje SQL personalizado de YFERA. Su objetivo es transformar comandos de manipulación de tablas en un AST que el GeneradorSQL pueda ejecutar sobre la base de datos del proyecto.

1. Información General

Tipo: Analizador Gramatical (LALR).

Responsabilidad: Definir la sintaxis para creación de tablas (TABLE), consultas (SELECT), inserciones (INSERT), actualizaciones (UPDATE) y eliminaciones (DELETE).

2. Componentes del Lexer

El analizador léxico está diseñado para reconocer palabras clave de bases de datos y manejar tipos de datos específicos:

Gestión de Strings: Implementa un estado inclusivo <string> para capturar cadenas entre comillas dobles, permitiendo caracteres de escape como \n, \" y \\.

Comentarios: Soporta tres tipos de comentarios: multilinea (/ * */), de línea simple (#) y el estándar SQL (--).

Tipos de Datos: Reconoce explícitamente int, float, string, boolean y char.

3. Estructura de la Gramática (AST)

El parser genera objetos JSON estructurados para cada comando:

Create Table: Almacena el nombre de la tabla y una lista de definiciones de columnas (nombre + tipo).

Select: En este lenguaje, el select está simplificado a la forma Tabla.Columna; para facilitar la extracción de datos hacia arreglos en el lenguaje .y.

Insert/Update: Utilizan una lista _asignaciones para mapear valores a columnas específicas. El Update requiere obligatoriamente la cláusula IN [id].

Delete: Sintaxis directa: NombreTabla DELETE [id];.

4. Manejo de Errores y Estado

Recuperación de Errores: Utiliza el token error seguido de PUNTO_COMA para recuperarse de sentencias mal formadas sin detener el análisis completo del script.

Comunicación Externa: Define métodos globales (reiniciarErrores, obtenerErrores) para que el GeneradorSQL.js pueda extraer los fallos léxicos y sintácticos acumulados durante el proceso de análisis.

5. Reglas de Valor

Los valores son convertidos automáticamente a sus tipos nativos de JavaScript durante la reducción:

NUMERO_ENTERO -> parseInt

NUMERO_DECIMAL -> parseFloat

TRUE / FALSE -> boolean real.

GeneradorSQL (Controlador de Persistencia)

Esta clase actúa como el orquestador del módulo SQL. Se encarga de limpiar el estado del parser, capturar errores de sintaxis personalizados y, opcionalmente, disparar la ejecución de los comandos si el análisis fue exitoso.

1. Información General

Tipo: Orquestador de Módulo / Fachada.

Responsabilidad: Coordinar el análisis léxico/sintáctico, gestionar la recuperación de errores y delegar la ejecución al EjecutorSQL.

2. Gestión Personalizada de Errores Sintácticos

Una característica clave es la sobreescritura del método `parseError` de Jison:

Localización Precisa: Extrae línea y columna exactas desde el objeto hash del parser.

Sugerencias Inteligentes: Si el parser detecta tokens esperados, los limpia y los presenta al usuario (ej: "Se esperaba: IDENTIFICADOR, PUNTO_COMA. Se encontró: 'TABLE'"), limitando la sugerencia a los primeros 5 tokens para mantener la claridad.

Integración: Registra estos fallos directamente en el listado de errores del módulo `sqlModulo`.

3. Flujo de Análisis (analizar)

El método principal sigue un pipeline estricto:

Reinicio: Limpia errores de sesiones anteriores.

Parseo: Llama al `sqlModulo.parse`. Si el código es inválido, el proceso de ejecución se detiene automáticamente.

Recolección: Filtra y unifica los errores léxicos y sintácticos.

Ejecución Condicional: Solo si no hay errores de sintaxis y la opción ejecutar es verdadera, instancia al `EjecutorSQL` para impactar la base de datos.

4. Interfaz de Salida

El método retorna un objeto estandarizado que es consumido tanto por el `TraductorY` (para sentencias `execute` en variables) como por el `InvocadorFuncionY`:

exito: Booleano que indica si todo el proceso (incluida ejecución) terminó sin fallos.

ast: La representación arbórea de las sentencias.

resultados: Los datos obtenidos (especialmente en `SELECT`) o mensajes de confirmación de filas afectadas.

errores: Una lista plana de objetos de error con formato uniforme.

5. Dependencias Críticas

`sql.jison`: Provee la lógica de tokens y gramática.

`EjecutorSQL`: Provee la lógica de bajo nivel para manipular archivos o bases de datos (`SQLite/JSON`).

`ErrorYFERA`: Clase base para mantener la consistencia de reportes en todo el compilador.

EjecutorSQL (Motor de Base de Datos)

Esta clase se encarga de la gestión física de la base de datos del proyecto. Utiliza better-sqlite3 para garantizar velocidad y soporte de transacciones síncronas, ideales para un entorno de compilación.

1. Información General

Tipo: Motor de Ejecución / Capa de Datos.

Responsabilidad: Abrir/crear archivos .db, validar la existencia de esquemas (tablas y columnas) y ejecutar las operaciones CRUD.

2. Gestión de Conexión y Archivos

Aislamiento por Proyecto: La base de datos se busca en una ruta dinámica basada en el nombre del proyecto (rutaBaseProyectos/proyecto/database.db).

Ciclo de Vida Seguro: El método ejecutar envuelve toda la operación en un bloque try-finally para asegurar que la base de datos se cierre siempre, incluso si ocurre un error fatal.

3. Lógica de Mapeo (YFERA a SQLite)

Dado que SQLite tiene tipos limitados, el ejecutor realiza una traducción inteligente:

int y boolean → INTEGER (los booleanos se guardan como 0 o 1).

float → REAL.

string y char → TEXT.

ID Automático: Todas las tablas creadas incluyen automáticamente una columna id (INTEGER PRIMARY KEY AUTOINCREMENT), la cual está prohibida definir manualmente en el código YFERA para evitar conflictos.

4. Validaciones Semánticas en Tiempo de Ejecución

A diferencia del parser, el ejecutor realiza validaciones de estado real:

Integridad de Esquema: Antes de un INSERT, SELECT o UPDATE, verifica mediante PRAGMA table_info si las columnas y tablas existen realmente en el archivo .db.

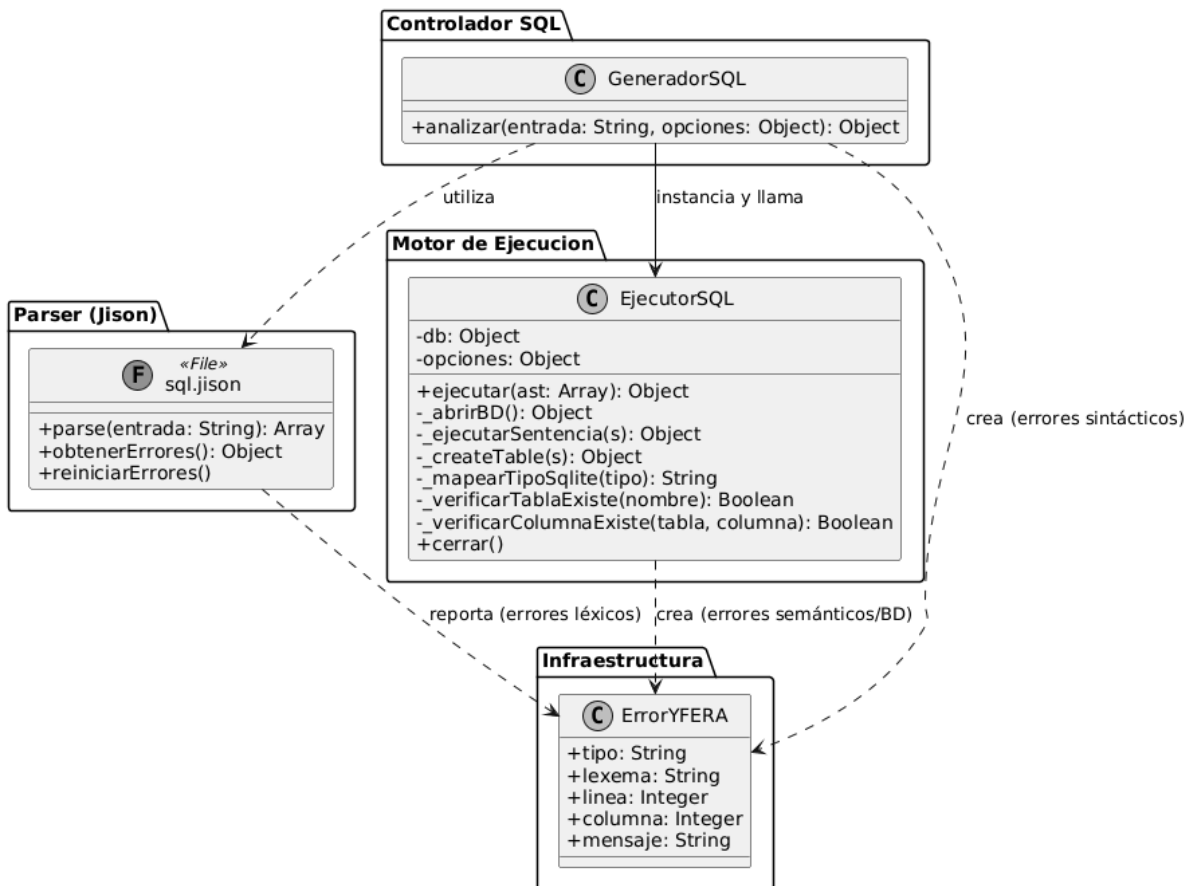
Nombres Duplicados: Evita la creación de tablas con columnas repetidas.

Manejo de IDs: En UPDATE y DELETE, reporta si la operación tuvo éxito pero no afectó filas (cuando el ID no existe), diferenciándolo de un error de sintaxis.

5. Seguridad y Robustez

Sentencias Preparadas: Utiliza db.prepare(...).run/all con placeholders (?) para prevenir inyecciones SQL, incluso si los datos vienen de formularios externos.

Identificadores Seguros: Envuelve nombres de tablas y columnas en comillas dobles (") para permitir el uso de palabras reservadas o caracteres especiales como identificadores.



Capa de Controladores (API REST)

Esta capa encapsula la lógica de negocio del servidor, orquestando los diferentes generadores para procesar código .comp, .styles, .sql y .y.

1. Información General

Tipo: Controladores de API (Patrón MVC).

Responsabilidad: Manejo de peticiones HTTP, validación de inputs y formateo de respuestas JSON.

2. Controlador de Componentes e Interface (ControladorComp & ControladorEstilos)

Son los puntos de entrada para la parte visual del proyecto:

ControladorComp: Retorna no solo el AST, sino también el HTML generado y la tabla de símbolos para que el frontend pueda renderizar la vista previa de los componentes.

ControladorEstilos: Un endpoint más ligero que valida la sintaxis de las hojas de estilo y reporta errores de diseño.

3. Controlador de Persistencia (ControladorSQL)

Diferencia claramente entre dos flujos de trabajo:

Analizar: Solo verifica que el código SQL sea sintácticamente correcto (útil para validación en tiempo real en editores).

Ejecutar: Requiere un nombre de proyecto para localizar la base de datos física en el workspace y aplicar los cambios.

4. Controlador de Lógica (ControladorY)

Es el controlador más complejo, ya que gestiona tanto el análisis como la ejecución de funciones específicas:

Analizar: Procesa el archivo .y para validar tipos, variables e imports.

Invocación Dinámica: Permite ejecutar una función específica dentro de un archivo .y (como un callback de un botón). Lee el archivo directamente del disco (fs.readFileSync) y utiliza el InvocadorFuncionY.

5. Gestión de Espacios de Trabajo (Workspaces)

Todos los controladores que requieren persistencia o lectura de archivos utilizan path.resolve para apuntar a una carpeta workspaces. Esto asegura que el servidor sea "multi-proyecto", aislando los archivos y bases de datos de cada usuario o aplicación.

ControladorWorkspace (Gestión de Proyectos)

Este controlador actúa como una interfaz entre las peticiones HTTP y el ServicioWorkspace. Su enfoque principal es la manipulación de directorios y archivos dentro del área de trabajo del servidor.

1. Información General

Tipo: Controlador de API / Gestión de Archivos.

Responsabilidad: Administrar el ciclo de vida de los proyectos (crear, listar, eliminar) y sus contenidos (archivos y carpetas).

2. Operaciones de Proyecto

Aislamiento: Todas las rutas se resuelven a partir de una rutaBase (generalmente la carpeta workspaces), asegurando que el servidor no acceda a archivos fuera de su zona permitida.

Gestión Global: Permite obtener la lista completa de proyectos disponibles para que el usuario pueda seleccionar su espacio de trabajo al iniciar la aplicación.

3. Operaciones de Archivo (CRUD de Disco)

Exploración: El método listarArchivos devuelve una estructura de "árbol" (generada por el servicio), ideal para alimentar componentes de File Explorer en el frontend.

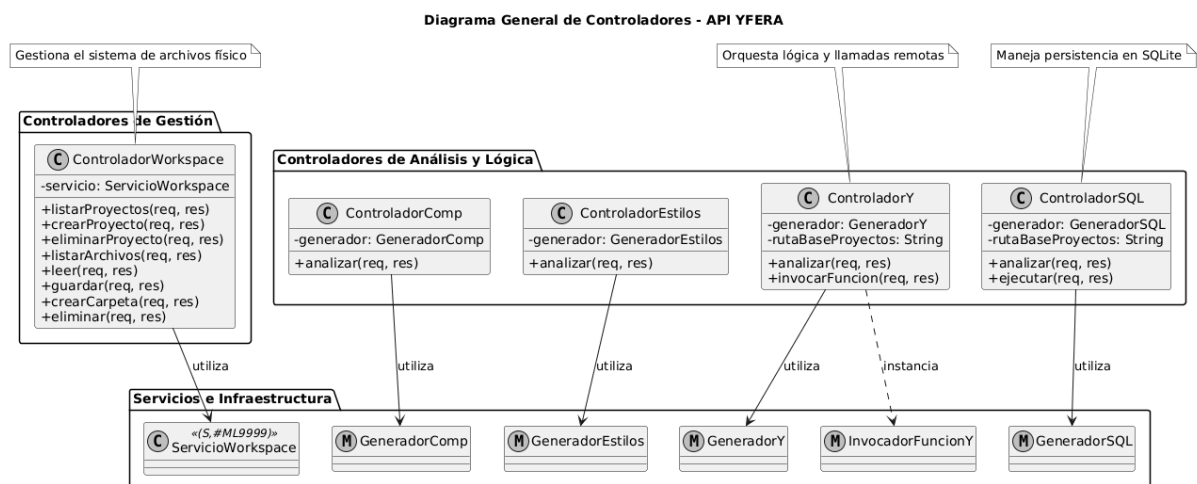
Persistencia: Gestiona la lectura (leer) y escritura (guardar) de código fuente (.comp, .styles, .y, .sql), sirviendo como el motor de "Guardado" del editor.

Organización: Permite la creación de carpetas internas para organizar los componentes del compilador.

4. Seguridad y Validación

Tipado de Datos: Valida estrictamente que los parámetros proyecto, ruta y contenido sean cadenas de texto para evitar errores de ejecución en el sistema de archivos (fs).

Manejo de Errores: Captura excepciones de permisos de disco o rutas no encontradas, devolviendo estados HTTP 400 (error de usuario), 404 (no encontrado) o 500 (error de servidor).



Servidor Express e Infraestructura de Rutas

El archivo index.js (o app.js) actúa como el núcleo del backend, configurando el entorno de ejecución de red y orquestando los sub-módulos de rutas.

1. Información General

Tipo: Servidor Web API REST (Express).

Responsabilidad: Levantar el servicio, configurar Middlewares de seguridad y parseo, y despachar peticiones a los controladores correspondientes.

Puerto: 3000 (por defecto).

2. Configuración de Middlewares

CORS: Habilitado para permitir que el IDE (frontend) se comuniquen con el backend sin bloqueos de seguridad del navegador.

JSON Parser: Configurado con un límite: '5mb', vital para recibir archivos de código fuente extensos o imágenes base64 si fuera necesario.

Estado del API: Incluye un endpoint /api/estado para pruebas rápidas de conectividad (Health Check).

3. Estructura de Endpoints (API)

La API queda organizada de forma modular bajo el prefijo /api:

Prefijo	Responsabilidad	Operaciones Clave
/workspace	Persistencia en disco	Listar proyectos, leer/guardar archivos, crear carpetas.
/comp	Frontend YFERA	Analizar sintaxis y generar HTML de componentes.
/estilos	Diseño	Validar sintaxis CSS/Styles.
/y	Lógica de negocio	Analizar tipos y ejecutar funciones remotas (Invocador).
/sql	Persistencia de datos	Ejecutar queries CRUD sobre la base de datos del proyecto.

Estructura de Rutas y Punto de Entrada

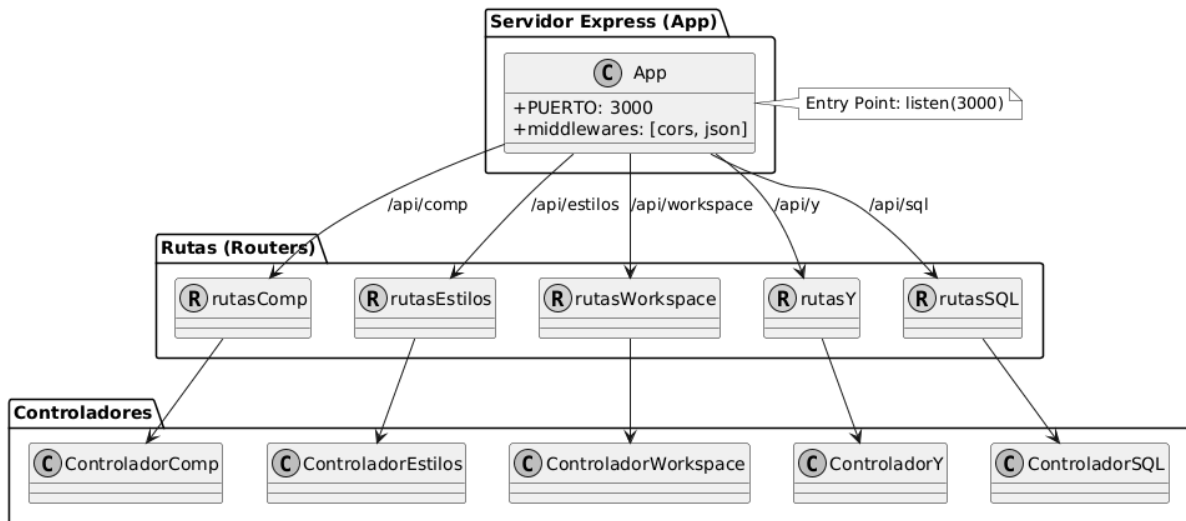
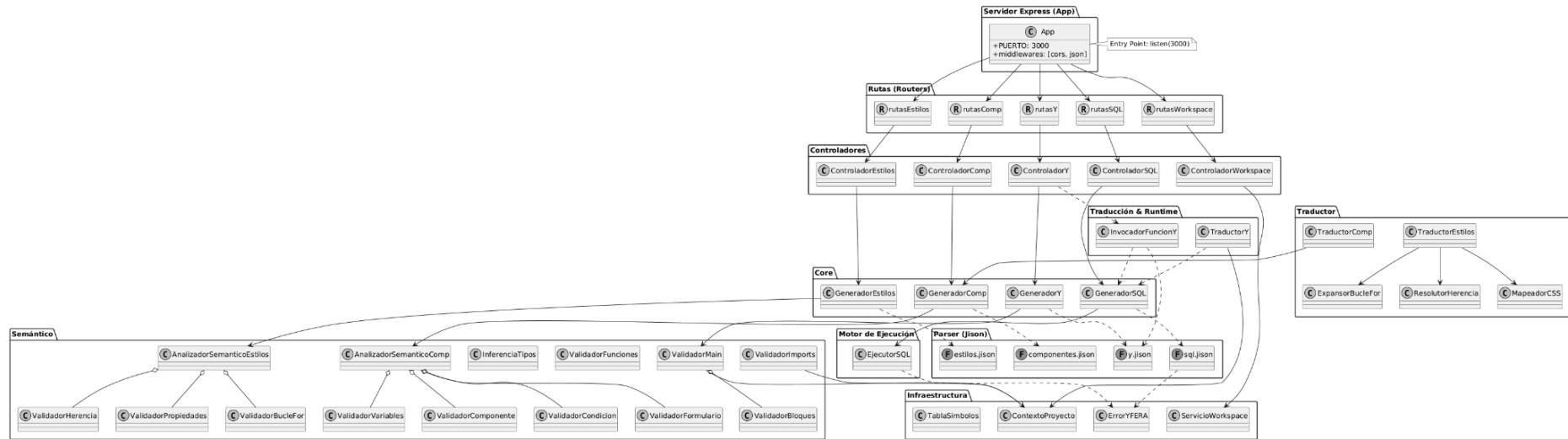


Diagrama general de clases

Diagrama General de Clases - Sistema YFERA



Flujo General de Peticiones - Sistema YFERA



Tecnologías utilizadas

Tecnologías de Base y Entorno

El núcleo del sistema se apoya en el ecosistema de JavaScript para garantizar asincronía y alto rendimiento en el procesamiento de archivos.

Node.js: Entorno de ejecución para la capa de backend. Se eligió por su eficiencia en operaciones de entrada/salida (I/O) y su amplio ecosistema de paquetes.

Express.js: Framework web utilizado para construir la API REST. Gestiona el ruteo, los middlewares de seguridad (CORS) y el procesamiento de peticiones JSON.

Better-SQLite3: Motor de base de datos síncrono para Node.js. Se utiliza para la persistencia de datos de los proyectos debido a su velocidad y facilidad para manejar archivos .db individuales por proyecto.

Ingeniería de Lenguajes (Compilación)

Jison: Generador de analizadores sintácticos (Parsers) basado en Bison/Flex. Es la herramienta principal para definir las gramáticas LALR.

Lexer: Segmenta el código fuente en tokens (palabras clave, identificadores, símbolos).

Parser: Construye el AST (Abstract Syntax Tree) validando la jerarquía y las reglas gramaticales.

Patrón Interpreter: El sistema utiliza intérpretes para recorrer los AST generados, permitiendo la ejecución de lógica dinámica en tiempo real sin necesidad de compilar a código máquina binario.

Gestión de Infraestructura y Archivos

Tecnologías encargadas de que el servidor se comporte como un entorno de desarrollo real.

File System (fs) & Path: Módulos nativos de Node.js utilizados para la manipulación física de archivos en el servidor.

Patrón Singleton / Service: El ServicioWorkspace centraliza toda la lógica de acceso a disco, aplicando medidas de seguridad como la prevención de Path Traversal.

REST API: Estilo de arquitectura para la comunicación entre el IDE (Cliente) y el compilador (Servidor), utilizando métodos estándar (GET, POST, DELETE).

Comandos utilizados:

cd Backend

npm install express

npm install jison

npm install better-sqlite3

npm install cors

npm install archiver@5

node src/[index.js](#)

comandos de compilacion

npx jison src/analizadores/estilos/estilos.jison -o src/analizadores/estilos/estilos.js

npx jison src/analizadores/comp/comp.jison -o src/analizadores/comp/comp.js

npx jison src/analizadores/y/y.jison -o src/analizadores/y/y.js

npx jison src/analizadores/sql/sql.jison -o src/analizadores/sql/sql.js

Frontend

ng serve

Gramatica estilos

archivo

```
__ definiciones
|  __ definiciones (vacio)
|  __ definicion
|      __ estilo
|          __ IDENTIFICADOR ("caja")
|          __ LLAVE_IZQ ("{" )
|          __ propiedades
|              __ lista_propiedades
|                  __ lista_propiedades
|                      __ propiedad
|                          __ nombre_propiedad
|                          |  __ lista_identificadores
|                          |      __ IDENTIFICADOR ("color")
|                          |      __ IGUAL ("=")
|                          |      __ valores
|                          |          __ valor_item
|                          |              __ IDENTIFICADOR ("blue")
|                          |              __ PUNTO_COMA (";")
|                          __ propiedad
|                              __ nombre_propiedad
|                              |  __ lista_identificadores
|                              |      __ lista_identificadores
|                              |          __ IDENTIFICADOR ("text")
|                              |          __ IDENTIFICADOR ("size")
|                              |      __ IGUAL ("=")
|                              |      __ valores
|                              |          __ valor_item
|                              |              __ expresion
|                              |              __ ENTERO ("14")
|                              |      __ PUNTO_COMA (";")
|              __ LLAVE_DER ("}")
|  __ EOF
```

bucle for

archivo

```
__ definiciones
|  __ definiciones (vacio)
|  __ definicion
|      __ bucle_for
|          __ FOR_LOOP ("@for")
|          __ VARIABLE ("$i")
|          __ FROM ("from")
|          __ ENTERO ("1")
|          __ THROUGH ("through")
|          __ ENTERO ("3")
|          __ LLAVE_IZQ ("{" )
|          __ cuerpo_for
|              __ cuerpo_for (vacio)
|              __ estilo_for
|                  __ nombre_for
|                      __ IDENTIFICADOR ("titulo")
|                      __ MENOS ("-")
|                      __ VARIABLE ("$i")
|                      __ LLAVE_IZQ ("{" )
|                      __ propiedades
|                          __ lista_propiedades
|                              __ propiedad
|                                  __ nombre_propiedad
|                                      __ lista_identificadores
|                                          __ lista_identificadores
|                                              __ IDENTIFICADOR ("text")
|                                              __ IDENTIFICADOR ("size")
|                                          __ IGUAL ("=")
|                                  __ valores
|                                      __ valor_item
|                                          __ expresion
|                                              __ VARIABLE ("$i")
|                                      __ PUNTO_COMA (";")
|                                  __ LLAVE_DER ("}")
|          __ LLAVE_DER ("}")
__ EOF
```

propiedades

propiedad

```
|__ nombre_propiedad
|  |__ lista_identificadores
|    |__ lista_identificadores
|    |  |__ IDENTIFICADOR ("background")
|    |  |__ IDENTIFICADOR ("color")
|__ IGUAL ("=")
|__ valores
|  |__ valor_item
|    |__ RGB ("rgb")
|    |__ PAR_IZQ "("
|    |__ ENTERO ("0")
|    |__ COMA (",")
|    |__ ENTERO ("100")
|    |__ COMA (",")
|    |__ ENTERO ("255")
|    |__ PAR_DER (")")
|__ PUNTO_COMA (";")
```

Gramatica de lenguaje COMP

T ::= "T"
IMG ::= "IMG"
INT ::= "int"
FLOAT ::= "float"
STRING_T ::= "string"
BOOLEAN ::= "boolean"
CHAR ::= "char"
FUNCTION ::= "function"
IF ::= "if"
ELSE ::= "else"
SWITCH ::= "Switch"
CASE ::= "case"
DEFAULT ::= "default"
FOR ::= "for"
EACH ::= "each"
TRACK ::= "track"
EMPTY ::= "empty"
FORM ::= "FORM"
INPUT_TEXT ::= "INPUT_TEXT"
INPUT_NUMBER ::= "INPUT_NUMBER"
INPUT_BOOL ::= "INPUT_BOOL"
SUBMIT ::= "SUBMIT"
TRUE ::= "true"
FALSE ::= "false"
ID_PROP ::= "id"
LABEL_PROP ::= "label"
VALUE_PROP ::= "value"
COR_DOBLE_IZQ ::= "["
COR_DOBLE_DER ::= "]"
COR_IZQ ::= "["
COR_DER ::= "]"
PAR_IZQ ::= "("
PAR_DER ::= ")"
LLAVE_IZQ ::= "{"
LLAVE_DER ::= "}"
MAYOR_IGUAL ::= ">="
MENOR_IGUAL ::= "<="
IGUAL_IGUAL ::= "=="
DIFERENTE ::= "!="
AND ::= "&&"
OR ::= "||"
MENOR ::= "<"
MAYOR ::= ">"
COMA ::= ","
PUNTO_COMA ::= ";"
DOS_PUNTOS ::= ":"

MAS ::= "+"
 MENOS ::= "-"
 POR ::= "*"
 DIVIDIDO ::= "/"
 MODULO ::= "%"
 NEGACION ::= "!"
 IGUAL ::= "="
 NUMERO ::= [0-9]+ ("." [0-9]+)?
 REFERENCIA ::= "@" [a-zA-Z_] [a-zA-Z0-9_]*
 VARIABLE ::= "\$" [a-zA-Z_] [a-zA-Z0-9_]*
 IDENTIFICADOR ::= [a-zA-Z_] [a-zA-Z0-9_-]*
 CADENA ::= "\"" caracter* "\""
 COMENTARIO ::= "/" * "/" (ignorado)

programa

```

__ lista_componentes
|  __ componente
|    __ IDENTIFICADOR ("saludo")
|    __ PAR_IZQ "("
|    __ parametros_opt
|    |  __ ε
|    __ PAR_DER (")")
|    __ LLAVE_IZQ "{"
|    __ elementos_opt
|    |  __ elementos
|    |    __ elemento
|    |      __ texto
|    |        __ T ("T")
|    |        __ PAR_IZQ "("
|    |        __ CADENA ("Hola mundo")
|    |        __ PAR_DER (")")
|    __ LLAVE_DER ("}")
__ EOF
  
```

programa

```
|__ lista_componentes
|  |__ componente
|    |__ IDENTIFICADOR ("card")
|    |__ PAR_IZQ "("
|    |__ parametros_opt
|    |  |__ parametros
|    |    |__ parametro
|    |      |__ tipo
|    |        |__ STRING_T ("string")
|    |        |__ IDENTIFICADOR ("titulo")
|    |__ PAR_DER (")")
|    |__ LLAVE_IZQ "{"
|    |__ elementos_opt
|    |  |__ elementos
|    |    |__ elemento
|    |      |__ seccion
|    |        |__ MENOR ("<")
|    |        |__ lista_estilos
|    |        |  |__ IDENTIFICADOR ("mi-estilo")
|    |        |__ MAYOR (">")
|    |        |__ COR_IZQ "["
|    |        |__ elementos_opt
|    |        |  |__ elementos
|    |        |    |__ elemento
|    |        |      |__ texto
|    |        |        |__ T ("T")
|    |        |        |__ PAR_IZQ "("
|    |        |        |__ CADENA ("Titulo: $titulo")
|    |        |        |__ PAR_DER (")")
|    |        |__ COR_DER ("]")
|    |__ LLAVE_DER ("}")
|__ EOF
```

Gramatica Lenguaje Y

IMPORT ::= "import"
FUNCTION ::= "function"
MAIN ::= "main"
EXECUTE ::= "execute"
LOAD ::= "load"
INT ::= "int"
FLOAT ::= "float"
STRING_T ::= "string"
BOOLEAN ::= "boolean"
CHAR ::= "char"
IF ::= "if"
ELSE ::= "else"
SWITCH ::= "switch"
CASE ::= "case"
DEFAULT ::= "default"
WHILE ::= "while"
DO ::= "do"
FOR ::= "for"
BREAK ::= "break"
CONTINUE ::= "continue"
TRUE ::= "true" | "True"
FALSE ::= "false" | "False"
IGUAL_IGUAL ::= "=="
DIFERENTE ::= "!="
MAYOR_IGUAL ::= ">="
MENOR_IGUAL ::= "<="

AND ::= "&&"
OR ::= "||"
INCREMENTO ::= "++"
DECREMENTO ::= "--"

IGUAL ::= "="
MAYOR ::= ">"
MENOR ::= "<"
MAS ::= "+"
MENOS ::= "-"
POR ::= "*"
DIVIDIDO ::= "/"
MODULO ::= "%"

NEGACION ::= "!"
COR_IZQ ::= "["
COR_DER ::= "]"
PAR_IZQ ::= "("
PAR_DER ::= ")"
LLAVE_IZQ ::= "{"
LLAVE_DER ::= "}"
COMA ::= ","

PUNTO_COMA ::= ";"
DOS_PUNTOS ::= ":"
REFERENCIA ::= "@" [a-zA-Z_] [a-zA-Z0-9_]*
NUMERO_DECIMAL ::= [0-9]+ "." [0-9]+
NUMERO_ENTERO ::= [0-9]+
IDENTIFICADOR ::= [a-zA-Z_] [a-zA-Z0-9_]*
CARACTER ::= "'" [^\n] "'"
CADENA_SQL ::= "`" [^`]* "`"
CADENA ::= "\" caracter* \""
COMENTARIO_LINEA ::= "#" . * (ignorado)
COMENTARIO_BLOQUE ::= "/" * . * "/" (ignorado)

programa

```
|__ lista_declaraciones
|  |__ lista_declaraciones
|  |  |__ lista_declaraciones
|  |  |  |__ lista_declaraciones
|  |  |  |  |__ declaracion
|  |  |  |  |__ import_decl
|  |  |  |  |  |__ IMPORT ("import")
|  |  |  |  |  |__ CADENA ("./componentes.comp")
|  |  |  |  |  |__ PUNTO_COMA (";")
|  |  |  |__ declaracion
|  |  |  |  |__ variable_decl
|  |  |  |  |  |__ tipo
|  |  |  |  |  |  |__ INT ("int")
|  |  |  |  |  |  |__ IDENTIFICADOR ("contador")
|  |  |  |  |  |  |__ IGUAL ("=")
|  |  |  |  |  |  |__ expresion
|  |  |  |  |  |  |  |__ expresion_and
|  |  |  |  |  |  |  |  |__ ... (cae a expresion_primaria)
|  |  |  |  |  |  |  |  |__ NUMERO_ENTERO ("5")
|  |  |  |  |  |  |  |__ PUNTO_COMA (";")
|  |  |__ declaracion
|  |  |  |__ function_decl
|  |  |  |  |__ FUNCTION ("function")
|  |  |  |  |__ IDENTIFICADOR ("reiniciar")
|  |  |  |  |__ PAR_IZQ "("
|  |  |  |  |__ parametros_opt
|  |  |  |  |  |__ ε
|  |  |  |  |  |__ PAR_DER (")")
|  |  |  |  |  |__ LLAVE_IZQ "{"
|  |  |  |  |  |__ cuerpo_funcion_opt
|  |  |  |  |  |  |__ cuerpo_funcion
|  |  |  |  |  |  |  |__ sentencia_funcion
|  |  |  |  |  |  |  |  |__ load_stmt
|  |  |  |  |  |  |  |  |__ LOAD ("load")
|  |  |  |  |  |  |  |  |__ CADENA ("./main.y")
|  |  |  |  |  |  |  |  |__ PUNTO_COMA (";")
|  |  |  |  |  |__ LLAVE_DER ("}")
|  |__ declaracion
|  |  |__ main_decl
|  |  |  |__ MAIN ("main")
|  |  |  |__ LLAVE_IZQ "{"
|  |  |  |__ cuerpo_main_opt
|  |  |  |  |__ cuerpo_main
|  |  |  |  |  |__ sentencia_main
|  |  |  |  |  |  |__ invocacion_componente
|  |  |  |  |  |  |  |__ REFERENCIA ("@header")
|  |  |  |  |  |  |  |__ PAR_IZQ "("
|  |  |  |  |  |  |  |__ argumentos_opt
|  |  |  |  |  |  |  |  |__ ε
|  |  |  |  |  |  |  |  |__ PAR_DER (")")
|  |  |  |  |  |  |  |  |__ PUNTO_COMA (";")
|  |  |  |  |  |__ LLAVE_DER ("}")
|__ EOF
```

Gramatica SQL

TABLE ::= "TABLE"
COLUMNS ::= "COLUMNS"
DELETE ::= "DELETE"
IN ::= "IN"
INT ::= "int"
FLOAT ::= "float"
STRING_T ::= "string"
BOOLEAN ::= "boolean"
CHAR ::= "char"
TRUE ::= "true" | "True"
FALSE ::= "false" | "False"
IGUAL ::= "="
COMA ::= ","
PUNTO_COMA ::= ";"
PUNTO ::= "."
COR_IZQ ::= "["
COR_DER ::= "]"
NUMERO_DECIMAL ::= [0-9]+ "." [0-9]+
NUMERO_ENTERO ::= [0-9]+
IDENTIFICADOR ::= [a-zA-Z_] [a-zA-Z0-9_]*
CARACTER ::= "" [^\n] ""
CADENA ::= "" caracter* ""
COMENTARIO_BLOQUE ::= "/"* . * "/" (ignorado)
COMENTARIO_LINEA ::= "#" . * (ignorado)
COMENTARIO_SQL ::= "--" . * (ignorado)

arbol para un CREATE

programa

```
__ lista_sentencias
|  __ sentencia
|    __ create_table
|      __ TABLE ("TABLE")
|      __ IDENTIFICADOR ("pokemons")
|      __ COLUMNS ("COLUMNS")
|      __ lista_columnas
|        __ lista_columnas
|          __ lista_columnas
|            __ columna
|              __ IDENTIFICADOR ("nombre")
|              __ IGUAL ("=")
|              __ tipo
|                __ STRING_T ("string")
|              __ COMA (",")
|              __ columna
|                __ IDENTIFICADOR ("nivel")
|                __ IGUAL ("=")
|                __ tipo
|                  __ INT ("int")
|              __ COMA (",")
|              __ columna
|                __ IDENTIFICADOR ("hp")
|                __ IGUAL ("=")
|                __ tipo
|                  __ FLOAT ("float")
|              __ PUNTO_COMA (";")
|  __ EOF
```